AVA_DB RAG ARCHITECTURE PROPOSAL

💡 Overview:
A two-stage system that interprets user queries and generates consistently structured reports regardless of the subject.
Workflow structure:
User Query
↓
Stage 1: Intent Router
↓
Stage 2: Structured Report Generator
↓
Formatted Output

The system separates **query interpretation** from **report generation**, allowing the chatbot to dynamically retrieve or summarize data while maintaining a standardized report structure.

▼ 1 Objective

Build a chatbot experience powered by **Ava** that can answer buyer performance questions with:

- **Accuracy on-demand** (when users ask for exact, time-bounded results)
- **Low operational overhead** (when users ask for trends/general performance)
- **Consistent, structured output formatting** across buyers and time periods (same report layout, different data)

This architecture combines:

- **Intent routing** to choose between live DB queries vs semantic retrieval
- A **canonical report template system** using block-based rendering and consistent schemas

▼ 2 System Architecture

The system architecture aims to expand the two-stage workflow described earlier into a set of modular components responsible for query interpretation, data retrieval, and structured report generation.

The architecture is designed to support both **precise data retrieval** and **semantic trend analysis** while maintaining a consistent output format.

▼ 2.1 High-Level Architecture Flow

```
User Query
↓
Ava Chat Interface (session + streaming)
↓
Stage 1: Intent Router
↓
Mode Decision (Precise vs Trend vs Hybrid)
↓
Data Retrieval Layer
├─ Precise → Database Query Engine
└─ Trend   → Semantic Retrieval Engine (Vector DB / RAG)
↓
Normalization Layer (canonical report schema)
↓
Stage 2: Structured Report Generator (Template)
↓
Ava LLM Synthesis (fills template using normalized data)
↓
Formatted Output (consistent structure)
```

Each component is responsible for a specific step in transforming a user question into a structured analytical report.

▼ 2.2 Chat Interface

The Chat Interface will be responsible for:

- Receiving user queries
- Maintaining session context
- Streaming output responses back to the user

This layer is the interaction boundary between the end user and the backend decision + retrieval system.

▼ 2.3 Stage 1 — Intent Router (Decision Layer)

The Intent Router interprets the user's request and outputs a structured "execution plan" describing:

- **Entity**: who/what the user is asking about (e.g., buyer Bob, buyer Mary)
- **Timeframe**: the period being requested (e.g., March 2023, Year 2000, overall)
- **Intent type**: whether the user needs **precise answers** or **trend-level insights**
- **Mode**: Precise, Trend, or Hybrid

Routing Requirement (Core)

After intent routing:

- **If Precise:** the system must query the database to return exact, time-bounded results.
- **If Trend:** the system must use semantic retrieval (vector DB) to avoid querying the database for broad analytical questions.
- **If Hybrid(EDGE CASE FOR SECOND PROTOTYPE):** the system uses semantic retrieval for narrative context and database querying only for any required exact metrics.

This ensures the system balances **accuracy when required** with **efficiency for general questions**.

▼ 2.4 Data Retrieval Layer (Two Retrieval Paths)

▼ A) Database Query Engine (Precise Path)

Triggered when the user requests accurate/time-bounded metrics (e.g., "Bob in March 2023").

<u>Responsibilities and workflow:</u>

- Fetch relevant records from the database using structured retrieval methods (e.g., by_id, by_field, etc.)

- Aggregate metrics into a consistent internal structure
- Return results in a canonical format for the report generator

This retrieval path prioritizes precision and accurate results.

▼ B) Semantic Retrieval Engine (Trend or Semantic retrieval)

Triggered when the user requests general performance trends or insights (e.g., “Bob overall trend”).

Responsibilities and workflow:

- Retrieve relevant “performance summary documents” from a vector database
- These documents represent precomputed summaries (buyer-month, buyer-year, rolling trends, anomaly notes)
- Embeddings for documents and queries are generated through Ava’s embeddings endpoint

This retrieval path prioritizes low operational overhead and avoids repeated full database pull.

▼ 2.5 Normalization Layer (Canonical Data Contract)

This is where the template tool requirement comes into picture(based on the back and forth answering and not the drag-drop tool) where the retrieved results must be converted into a single canonical schema that the report generator expects.

Example:

```
{
  "buyer": { "first_name": "John","last_name": "Do
e","id": 123 },
  "period": { "start": "2023-03-01", "end": "2023-03-3
1" },
  "kpis": { "leads": 0, "tasks": 0, "pickups": 0 },
  "highlights": [],
  "risks": [],
  "actions": [],
  "source_mode": "precise"
}
```

▼ 2.6 Stage 2 — Structured Report Generator (Template + Ava Synthesis)

Stage 2 enforces a consistent report structure using a canonical template approach, this is more user facing results as these results will be sent directly to the chat interface.

This ensures that:

- “How did Bob perform in March 2023?”
- “How did Mary perform in Year 2000?”

produce the same report layout, with only the data values and insights changing.

This aligns with the canonical template + block-based structure described in the templating system design.

▼ 2.7 Output Format

This is where the synthesized result comes into picture where the structured report must be sent back to the chat interface, into human readable format with desired structured reports)

▼ 3 Intent Routing Logic

This section defines how the system decides whether a user request should be handled via an **exact database pull** (Precise Mode), **semantic retrieval** (Trend Mode). The intent router outputs a structured execution plan that downstream components can execute deterministically.

3.1 Router Inputs and Outputs

Input

- User query (natural language)
- Conversation context (optional, to further streamline workflow)
- Known entities (buyers, etc.) (optional, to further streamline workflow)

Output (Execution Plan)

The main aim of intent execution path is to produce a JSON format categorization of the result, so what we are aiming for must look something

like this:

```
{
  "intent": "buyer_performance",
  "entity": { "type": "buyer", "name": "Bob", "id": "res
olved_later" },
  "timeframe": { "start": "2023-03-01", "end": "2023-03-
31", "granularity": "month" },
  "mode": "precise",
  "report_template_id": "buyer_performance_v1",
  "metrics_requested": ["kpis", "highlights", "risks",
"actions"]
}
```

*This is an example which will modify in the future, the metrics required and report template ID is a projected path but can change accordingly.

▼ 3.2 Mode Definitions

Precise Mode (Database-backed)

Goal: return exact, time-bounded results.

Triggered when the user request includes:

- a specific period (“March 2023”, “between Jan and Feb 2024”, “last week”)
- exactness language (“exact”, “how many”, “total”, “list”, “show all records”)
- requests for itemized outputs (records, IDs, lists)

Output expectations:

- structured metrics computed from database records
- no guesswork; values must be reproducible

Trend Mode (Semantic/RAG-backed)

Goal: answer general questions without repeatedly querying the database.

Triggered when the user request is:

- broad (“overall”, “in general”, “trend”, “how is X doing”)
- focused on interpretation (“patterns”, “insights”, “drivers”, “what changed”)
- comparative at a high level without requiring exact numeric proof

Output expectations:

- narrative + summarized patterns
- may reference time windows, but not presented as exact totals unless anchored by stored summaries

▼ 3.3 Routing Decision Rules

The router uses a deterministic rule set. A simple and explainable approach is recommended for early iterations.

Rule Group A — Strong Precise Signals

Route to **Precise** if the query includes any of:

- explicit timeframe (month/year/date range)
- “exact”, “total”, “count”, “how many”, “show me”, “list”, “records”
- audit-style phrasing (“ground truth”, “accurate numbers”, “verify”)

Rule Group B — Strong Trend Signals

Route to **Trend** if the query includes:

- “overall”, “in general”, “trend”, “pattern”, “insights”, “summary”
- “what’s driving”, “why”, “how has it changed”
- no explicit narrow timeframe requiring exact totals

▼ 3.4 Entity and Timeframe Extraction

Entity Extraction

The router attempts to identify the subject:

- name (“Bob”, “Mary”)
- role (“buyer”, “lead”)

If multiple buyers are referenced:

- treat as a comparison intent

- either use a comparison template or return multiple reports using the same template.

Timeframe Extraction

The router converts natural language time requests into:

- `start_date`
- `end_date`
- `granularity` (month/year/custom range)

Examples:

- "March 2023" → `granularity=month`
- "Year 2000" → `granularity=year`
- "last 30 days" → `granularity=range`

▼ 3.5 Routing Examples (Query → Mode → Action)

User Query	Mode	System Action
"How did Bob perform in March 2023?"	Precise	DB pull for March 2023, compute KPIs
"Give me the exact number of leads Bob had in March 2023."	Precise	DB pull + exact count
"How is Bob doing overall?"	Trend	Vector retrieval of Bob summaries + synthesize
"What's Bob's performance trend this year?"	Trend	Prefer semantic summaries; Think of either Hybrid(potential) if exact month totals requested
"Compare Bob and Mary overall."	Trend	Retrieve both sets of summaries + compare
"List all of Mary's pickups in 2000."	Precise	DB pull + itemized record listing
"Why did Bob's performance drop?"	Trend	Semantic retrieval + explanation
"Show me Bob's tasks completed last week."	Precise	DB pull for last week

User Query	Mode	System Action
“Give a high-level Q1 performance recap for Bob.”	Trend	Semantic retrieval of Q1/month summaries

▼ 3.6 Router Confidence and Fallback Behavior

To prevent misrouting:

- If timeframe is present but intent is unclear → default to **Precise** (safer for correctness).
- If entity cannot be resolved (e.g., multiple “Bob” matches) → ask a disambiguation question or use best match with a clear note.

▼ 4 Output Structuring Approach

This proposal does not implement the full templating system in the initial iteration. Instead, it establishes **two fixed structured response formats** (schemas) based on the routing outcome. Ava synthesizes responses using these schemas to ensure consistent structure across different entities and timeframes.

▼ 4.1 Design Goal

Ensure that answers for different buyers and timeframes follow a consistent structure.

Example:

- “How did Bob perform in March 2023?”
- “How did Mary perform in Year 2000?”

Both should follow the **same workflow and structure**, changing only the underlying content.

▼ 4.2 Structured Output by Mode

The system enforces structure at the **mode level**:

A) Precise Mode Output Format (DB-backed)

Precise Mode responses use a format optimized for **accuracy, auditability, and time-bounded reporting**.

Response shape (v1):

1. Request Summary (entity, timeframe, metric scope)
2. KPI Snapshot (exact numbers)
3. Supporting Breakdown (counts by category if applicable)
4. Notes / Limitations (what data was used, coverage caveats)
5. Next Suggested Question (optional)

Key principle: Precise answers prioritize “exactness” over interpretation.

B) Semantic Mode Output Format (Trend/RAG-backed)

Trend Mode responses use a format optimized for **interpretation, patterns, and narrative synthesis** while minimizing repeated database pulls.

Response shape (v1):

1. Executive Summary (high-level assessment)
2. Trend Narrative (what’s improving/declining)
3. Key Drivers / Explanations (likely causes from summaries)
4. Notable Periods / Highlights (time windows referenced from retrieved summaries)
5. Confidence / Provenance Note (e.g., “based on summarized history”)

Key principle: Semantic answers prioritize “insight and patterns” over exact totals.

▼ 4.3 Consistency Mechanism (How Ava stays structured without templates)

Instead of relying on a full template builder now, the system enforces structure through:

- **Mode-specific response schemas**
- A strict prompt contract to Ava:

  “Respond using the following section headings in the given order…”

This ensures:

- Bob and Mary follow the same structure (within a mode)
- March and Year 2000 follow the same structure (within a mode)

- Precise and Semantic have intentionally different formats

This approach is aligned with the longer-term direction of canonical structured templates, but is simpler to deliver in v1.

▼ 5 Pain Points, Risks, and Open Questions

This section captures known challenges early so they can be tracked and reviewed with the team lead.

▼ Key Pain Points/ Risk

▼ 1) Defining "Performance"

Risk: "performance" can mean different things (leads, tasks, pickups, sales, margin, etc.).

Impact: inconsistent KPIs, disagreement on what should be reported.

Mitigation: define a v1 KPI set explicitly (even if minimal), expand later.

▼ 2) Data Quality and Join Logic

Risk: Metrics may require joining across multiple tables/entities; missing or inconsistent keys can distort results.

Impact: precise answers may be "technically correct" but misleading.

Mitigation: add validation checks + report "coverage notes" in Precise output.

▼ 3) Routing Misclassification (Precise vs Trend)

Risk: user intent can be ambiguous ("How did Bob do last month?").

Impact: system might return a trend summary when user wanted exact totals, or vice versa.

Mitigation: conservative defaults:

- if timeframe is explicit → route Precise
- if user says "overall/trend" → route Semantic
- allow follow-up clarification ("Do you want exact totals or overall insights?")

▼ 4) Semantic Drift and Staleness

Risk:

trend summaries in the vector DB can become stale if not regenerated frequently.

Impact:

trend answers may lag behind reality.

Mitigation:

scheduled summary refresh pipeline; include “last updated” metadata in semantic responses.

▼ 5) Hallucination Risk in Semantic Mode

Risk: Ava may generate confident-sounding numbers that were not retrieved. Specially with its online surfing capabilities it can drift off topic heavily.

Impact: loss of trust.

Mitigation: hard constraint: semantic mode must avoid exact totals unless present in retrieved summary docs; include confidence/provenance notes.

▼ Questions to be answered along the way or before design.

→What are the KPIs that are the most important for the semantic searches to eventually embed into the vector database.

→How frequent should the semantic search values refresh and how to manage not available false back questions?

→Any desired initial template to weave answers from different context questions.

Once the database-backed metrics and semantic summary corpus are generated and embedded, we will enable **intent routing** and deliver a **v1 structured reporting experience** in the chat interface (mode-specific formatting for Precise vs Semantic). The subsequent milestone will focus on **prompt template refinement and hardening**, serving as the foundation for a future phase that introduces a **drag-and-drop UI template builder** for configurable report layouts.